

MODULE 25 · DOMAIN 6 - TROUBLESHOOTING, MONITORING AND OPTIMISATION

Liquid clustering and predictive optimisation

From partitioning to CLUSTER BY AUTO, and the maintenance Databricks now runs for you.

DOMAIN 6 of 7 EXAM WEIGHT 10% MODULE 25 of 36

Query speed depends on **how little data you can avoid reading**. Both features are about the same thing: who decides the layout, and who maintains it.

How we got here

01

Partitioning

Physical directories per value. Decided at creation, permanent.

02

Z-ORDER

Co-locates related data within files. Needs OPTIMIZE, and fixed columns.

03

Liquid clustering

CLUSTER BY. Keys can change without rewriting the table.

04

CLUSTER BY AUTO

Databricks picks and updates the keys from your query patterns.

THE DIRECTION OF TRAVEL

Each step removes a decision you had to get right in advance and could not easily change afterwards.

Why partitioning became a problem

- ◆ **Chosen at creation and effectively permanent.** Changing partition columns means rewriting the table.
- ◆ **High-cardinality columns produce thousands of tiny directories,** and the small files problem from module 3.
- ◆ **Low-cardinality columns barely prune anything.** Two partitions on a billion rows helps nobody.
- ◆ **A query that does not filter on the partition column gets no benefit at all.**

ON THE EXAM

Partitioning asks you to predict, at creation time, how a table will be queried for the rest of its life.

Liquid clustering

- ◆ **CLUSTER BY** replaces both partitioning and Z-ordering. You cannot combine it with either.
- ◆ **Clustering keys can be changed** without rewriting the table.
- ◆ **It handles skewed and high-cardinality columns** far better than partitioning does.
- ◆ **After altering clustering columns you must run OPTIMIZE** to feel the benefit.

ON THE EXAM

The headline feature is that the decision is reversible. A wrong clustering key is a change; a wrong partition column is a migration.

Automatic liquid clustering

- ◆ Set **CLUSTER BY AUTO** on a Unity Catalog managed table, and Databricks chooses the keys.
- ◆ **Predictive optimisation analyses the historical query workload** and selects the most effective keys.
- ◆ **It only changes keys when the benefit outweighs the overhead**, since clustering itself costs something.
- ◆ **Available for Unity Catalog managed tables only.**

ON THE EXAM

AUTO uses the current partition columns as its starting point, then adapts from there as query patterns emerge.

Two details about AUTO

- ◆ Reading and writing works on any runtime that supports liquid clustering.
- ◆ But intelligent key selection relies on metadata introduced in Databricks Runtime 15.4 LTS.
- ◆ You can enable it on any UC managed table; the heuristics decide whether clustering is worth it.
- ◆ Skip reasons are visible in Catalog Explorer's History tab, on the AUTO LIQUID row marked Not applied.

ON THE EXAM

If AUTO appears to do nothing, that may be the heuristics correctly deciding this table would not benefit. The History tab tells you why.

Databricks runs **OPTIMIZE, VACUUM and ANALYZE** for you, on Unity Catalog managed tables, deciding which tables need them and when.

The three operations

OPERATION	WHAT IT DOES
OPTIMIZE	Optimises file sizes, and triggers incremental clustering on clustered tables
VACUUM	Deletes data files no longer referenced by the table, reducing storage cost
ANALYZE	Collects and incrementally updates statistics, improving query planning

ON THE EXAM

These are the same three commands you would have scheduled by hand. The difference is that something decides when they are needed.

Where it does not run

- ◆ **Only on Unity Catalog managed tables.** External tables are excluded.
- ◆ **Not on streaming tables or materialized views** created in Databricks SQL or in pipelines.
- ◆ **Not available in all regions.**
- ◆ **It covers Delta Lake and Apache Iceberg** managed tables.

ON THE EXAM

That first exclusion is the strongest argument in the course for managed over external tables.

The Z-order interaction

- ◆ OPTIMIZE does not run ZORDER when executed by predictive optimisation.
- ◆ On tables that use Z-order, predictive optimisation ignores the Z-ordered files.
- ◆ So a Z-ordered table gets less benefit than you might expect from enabling it.
- ◆ This is a real reason to migrate to liquid clustering, rather than merely a stylistic preference.

ON THE EXAM

Enabling predictive optimisation on a Z-ordered table and expecting full maintenance is a specific, documented misunderstanding.

Whether it is already on

- ◆ Enabled by default for accounts created on or after 11 November 2024.
- ◆ Existing accounts have been enabled by a gradual rollout, which Databricks expected to complete by August 2026.
- ◆ There is a documented way to verify whether your account is enabled.
- ◆ Databricks recommends disabling scheduled OPTIMIZE jobs once predictive optimisation is running.

ON THE EXAM

That last point matters operationally. Leaving a nightly OPTIMIZE job in place means paying for maintenance twice.

The VACUUM warning

- ◆ VACUUM retention comes from `delta.deletedFileRetentionDuration`, which defaults to 7 days.
- ◆ So predictive optimisation will vacuum files older than that, and time travel beyond 7 days stops working.
- ◆ Set the property before enabling predictive optimisation if you need longer.
- ◆ `ALTER TABLE ... SET TBLPROPERTIES ('delta.deletedFileRetentionDuration' = '30 days')`

ON THE EXAM

Databricks flags this as a warning in its own documentation. Time travel you were relying on can quietly disappear.

Cost and visibility

- ◆ It runs on **serverless compute for jobs**, billed with a serverless jobs SKU.
- ◆ So it is **not free**, and it appears on your bill as compute you did not schedule.
- ◆ Track it with **system tables**, so the spend is attributable rather than mysterious.
- ◆ The trade is **usually favourable**, since it replaces jobs you were running anyway, and skips runs that were unnecessary.

ON THE EXAM

An unexplained serverless jobs line on the bill, shortly after enablement, is often this. It is worth recognising rather than investigating.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Partition columns turned out to be wrong	Liquid clustering — keys can change without a rewrite
Thousands of tiny partition directories	High-cardinality partitioning. Cluster instead.
Databricks should choose the clustering keys	CLUSTER BY AUTO, on a UC managed table
An external table is not being maintained	Predictive optimisation only runs on managed tables
A Z-ordered table gets little benefit	PO ignores Z-ordered files and does not run ZORDER
Time travel stopped working after enablement	VACUUM at the 7-day default retention
Unexplained serverless jobs cost appeared	Predictive optimisation, billed on a serverless jobs SKU

ON THE EXAM

Seven lines. The fourth, fifth and sixth are the three most likely to be tested, because each is an exclusion or a side effect.

A team partitioned a large events table by event_id when it was created three years ago. The table now has several million partition directories, most containing a single small file. Queries are slow and OPTIMIZE takes hours.

Almost all queries filter on event_date and customer_id, neither of which is the partition column.

What should they do?

- ☐ A Repartition the table by event_date, which most queries filter on
- ☐ B Migrate to liquid clustering with CLUSTER BY on event_date and customer_id
- ☐ C Add a Z-ORDER on event_date and customer_id alongside the existing partitioning
- ☐ D Enable predictive optimisation and let it resolve the layout

◆ PAUSE AND DECIDE

What should they do?

- A

 Repartition by event_date
Better than event_id, and it repeats the original mistake: a permanent choice made now, unchangeable when query patterns shift again.
- B

 Liquid clustering on the queried columns
Correct. It removes the partition explosion, targets the columns actually filtered on, and the keys can change later without a rewrite.
- C

 Z-ORDER alongside partitioning
It leaves several million directories in place, and predictive optimisation ignores Z-ordered files, so future maintenance is worse too.
- D

 Let predictive optimisation fix it
PO maintains a layout; it does not redesign one. It will not remove partitioning that was chosen at creation.

THE SIGNAL

Partitioning on a near-unique column is the textbook failure. The fix is not a better partition column — it is not partitioning.

A platform team enables predictive optimisation across the account. After a month they audit the results. Most tables are being optimised and vacuumed automatically as expected.

Three sets of tables are receiving nothing: the tables in their partner-shared external location, their Databricks SQL materialized views, and the streaming tables in their ingestion pipelines.

Why?

- ☐ A Predictive optimisation needs to be enabled per table as well as per account
- ☐ B It only runs on Unity Catalog managed tables, and does not run on streaming tables or materialized views
- ☐ C Those tables are too small to benefit, so the heuristics skipped them
- ☐ D The rollout has not reached those tables yet

◆ PAUSE AND DECIDE

Why?

A Enable it per table too

Does not explain why exactly these three categories were affected while everything else worked.

C Too small to benefit

The heuristics do skip tables, and that would not correlate so neatly with three distinct object types.

B Managed tables only; not streaming tables or MVs

Correct. Two documented exclusions cover all three groups: external tables are not managed, and streaming tables and MVs are excluded explicitly.

D The rollout has not reached them

The rollout is per account, not per table. Everything else in the account is working.

THE SIGNAL

Three symptoms, two documented exclusions. When affected objects share a type rather than a size, look for a categorical rule.

A team has predictive optimisation enabled. Their largest fact table uses Z-ORDER on three columns, maintained by a nightly OPTIMIZE job they wrote two years ago.

They remove the nightly job, expecting predictive optimisation to take over. Query performance degrades over the following weeks.

What happened?

- ☐ A Predictive optimisation does not run ZORDER, and ignores Z-ordered files, so the table is no longer being maintained
- ☐ B Predictive optimisation needs several months of query history before it acts
- ☐ C Removing the job also removed the table's statistics
- ☐ D The table should have been converted to an external table first

◆ PAUSE AND DECIDE

What happened?

- A

 PO does not run ZORDER and ignores Z-ordered files
Correct. Both statements are documented. Removing the nightly job left the Z-ordered table with no maintenance at all.
- B

 It needs months of history
Invented delay. PO queues maintenance based on writes and table state, not on a long warm-up period.
- C

 Removing the job removed statistics
Deleting a job does not delete table statistics. And ANALYZE is one of the operations PO would run anyway.
- D

 Convert to an external table
Backwards. PO runs only on managed tables, so converting to external would guarantee no maintenance at all.

THE SIGNAL

Databricks does recommend disabling scheduled OPTIMIZE jobs — but that advice assumes PO will actually maintain the table.

A finance team relies on being able to query a regulatory table as it stood on any day in the previous quarter, using Delta time travel. This has worked for two years.

Predictive optimisation is enabled on the account. Six weeks later, an auditor requests the table as at 40 days ago, and the query fails.

What happened, and what should have been done?

- A Time travel is not supported on managed tables once predictive optimisation is enabled
- B VACUUM ran at the default 7-day retention; delta.deletedFileRetentionDuration should have been set before enabling
- C OPTIMIZE rewrote the files, which invalidates all previous versions
- D The table needed to be converted to Iceberg to retain long-range time travel

◆ PAUSE AND DECIDE

What happened, and what should have been done?

- A

 Time travel is unsupported with PO
Time travel still works. What changed is how far back the files exist, which is a retention question rather than a support question.
- B

 VACUUM at the 7-day default; set retention first
Correct. Databricks flags this as a warning: set `delta.deletedFileRetentionDuration` before enabling, because vacuumed files are gone.
- C

 OPTIMIZE invalidates old versions
OPTIMIZE rewrites files and the old ones remain referenced by earlier versions until VACUUM removes them. VACUUM is the operation that ends time travel.
- D

 Convert to Iceberg
PO covers managed Delta and Iceberg alike, with the same VACUUM retention behaviour. Converting changes nothing here.

THE SIGNAL

For two years nobody ran VACUUM, so files accumulated and long time travel worked by accident. Enabling PO removed the accident.

Liquid clustering makes the layout decision **reversible**. Predictive optimisation makes the maintenance **automatic**. Both only fully apply to Unity Catalog managed tables.